

Izveštaj - obrazloženje rešenja

Projekat "Atoll" - veštačka inteligencija

Tim "X-Files"

Tara Milijić

Nikola Ristić

Nikola Ristovski

2025/26

Stanje igre i kreiranje inicijalnog i proizvoljnog stanja

Nikola Ristovski

Stanje igre u našem projektu predstavljeno je u vidu grafa, korišćenjem dictionary-a u python-u.

Ključevi su tuple objekti sa koordinatama tog polja u tabli, (A, 1) je na primer prvo polje u koloni A (polje na koje se može staviti kamenčić).

Polja koja su inicijalna ostrva su recimo (A, 0), (B, 0)... ali i cela LEFT i RIGHT kolona koje smo dodali za potrebu predstavljanja inicijalnih ostrva.

Od središnjeg slova (ako su slova od A do I, to je slovo E) kreće drugačije indeksiranje - svako slovo posle središnjeg ima početni indeks za 1 veći od prethodnog pa tako F ima prvo polje (F, 2), G ima (G, 3) - ovo su polja na kojima korisnik ima pravo da satavi kamen, a polja koja su inicijalna ostrva su idalje na vrhu kolone (F, 1), (G, 2) - najbolje se uočava sa slike na slajdovima.

Svaki ključ ima vrednost koja je lista sa 2 elementa - prvi je stanje polja, GREEN, RED ili None (None je slobodno) dok je drugi element lista suseda tog polja.

Primer:

```
stanje = {
    ('LEFT',1): ([...]),
    ...
    ('A',1): ('RED', [('A', 0), ('A', 2), ('B', 1), ('B', 2), ('LEFT', 1)]),
    ('A',2): (None, [('A', 1), ('A', 3), ('B', 2), ('B', 3), ('LEFT', 2)]),
    ...
    ('B', 1) : (None, [...])
}
```

Kako bismo znali opseg polja svake od kolona, kreirana je pomoćna funkcija `calculate_column_ranges(column_letters: list, board_size:int)` koja na osnovu veličine stranice table, i na osnovu toga da li je kolona pre kolone reprezentovane srednjim slovom (srednje slovo za kolone od A do I je E na primer), posle te kolone ili je jedna od ivičnih (LEFT ili RIGHT), računa prvi i poslednji indeks polja za tu kolonu - pa je tako za veličinu table $n = 5$, `limits['A'] = (0, 6)`, `limits['RIGHT'] = (6, 9)` itd.

Zatim, imamo i pomoćnu funkciju `calculate_column_ranges(column_letters: list, board_size:int)` koja je zadužena da pridoda svakom polju u grafu njegove susede. Prvo izračuna sve susede u obliku šestougla oko polja (ispod, iznad, kao i ispod i iznad u kolonama levo i desno od tog polja). Ako se desi da neki od njih ne postoji u listi polja koju smo prethodno kreirali (nema ga na tabli, nevalidan je), on se izbaci kasnije u comprehension-u jer je prepoznat po None vrednosti za ime kolone.

Poslednja pomoćna funkcija, `calculate_column_ranges(column_letters: list, board_size:int)` određuje koja polja su deo inicijalnih ostrvaca i da li pripadaju crvenim ili zelenim ostrvcima. To se opet određuje relativno veličini stranice table i tome koliko je polje u toj koloni udaljeno od kolone srednjeg slova (recimo E ako su slova od A do I). Naravno, uzimamo u obzir samo polja koja su baš prva i poslednja u datoj koloni, recimo A,0 i A,6 su takva. LEFT i RIGHT kolone su sastavljene isključivo od inicijalnih ostrva, pa je prva polovina LEFT kolone zeleno ostrvo, ostatak je crveno, i slično tome samo obrnuto važi za RIGHT kolonu.

Najzad, koristimo funkciju `create_initial_state(board_size: int)` da spojimo sve prethodne funkcije u jednu celinu i kreiramo inicijalno stanje sa ostrvima i praznim poljima. Na to se nadovezuje funkcija `create_arbitrary_state(green_cells:list, red_cells:list)` kojoj se prosledjuju liste zelenih i crvenih polja, kojima se zameni inicijalno kreirano stanje i tako dobijamo opciju kreiranja proizvoljnog stanja.

Unošenje poteza, postavljanje konfiguracija

Tara Milijić

Mehanizam unosa poteza realizovan je kroz skup pomoćnih funkcija koje obezbeđuju pravilnu interakciju korisnika sa logičkim stanjem igre, kao i validaciju svih unosa u skladu sa pravilima igre. Cilj ovih funkcija je da se spreče neispravni potezi i da se obezbedi stabilno i konzistentno ažuriranje stanja igre.

Osnovna funkcija za proveru ispravnosti poteza je `is_move_valid(game_state, position)`. Ona najpre proverava da li se zadato polje uopšte nalazi u strukturi grafa koja predstavlja stanje igre. Na taj način se onemogućava unos poteza van granica

table. Zatim se proverava da li je izabrano polje slobodno, odnosno da li mu je stanje **None**.

Za interakciju sa korisnikom implementirana je funkcija `input_a_move(game_state)`, koja omogućava tekstualni unos poteza u obliku koordinata (npr. C 3). Funkcija koristi petlju koja traje sve dok korisnik ne unese validan potez. Prilikom unosa, vrši se parsiranje kolone i reda, kao i konverzija reda u numerički tip. U slučaju neispravnog formata ili nevalidnog poteza, korisniku se ispisuje odgovarajuća poruka, a unos se ponavlja. Na ovaj način se obezbeđuje da se dalja logika igre izvršava isključivo nad ispravnim podacima.

Za konkretno ažuriranje stanja igre nakon unosa validnog poteza implementirana je funkcija `set_a_cell(state, position, current_player)`. Njena uloga je da izvrši promenu stanja izabranog polja u okviru postojeće strukture grafa. Funkcija prima trenutno stanje igre, koordinate polja koje je izabrano kao potez, kao i informaciju o igraču koji je na potezu. Na osnovu vrednosti parametra **current_player**, vrši se mapiranje simbola igrača na odgovarajuću boju polja u grafu. Ukoliko je trenutni igrač označen simbolom **X**, polje se postavlja u stanje **GREEN**, dok se u suprotnom slučaju postavlja u stanje **RED**.

Interakcija sa korisnikom realizovana je kroz posebne CLI funkcije

(`choose_game_mode_cli`, `choose_first_player_cli`, `choose_first_symbol_cli`, `choose_board_size_cli`), koje u petlji zahtevaju unos sve dok se ne dobije validna vrednost. Funkcija `choose_game_mode_cli()` omogućava izbor režima igre, čovek protiv čoveka ili čovek protiv računara. Ova informacija se kasnije koristi za određivanje toka igre i aktivaciju algoritama veštačke inteligencije. Funkcija `choose_first_player_cli()` omogućava izbor da li prvi potez ima čovek ili računar, dok `choose_first_symbol_cli()` definiše koji simbol započinje igru. Funkcija `choose_board_size_cli()` omogućava korisniku da interaktivno izabere veličinu table za igru, pri čemu su dozvoljene samo vrednosti 5, 7 ili 9. Njena svrha je da obezbedi validan unos pre nego što se igra pokrene i da automatski ažurira centralnu konfiguraciju igre **game_config**.

Sve CLI funkcije komuniciraju sa korisnikom i prikupljaju njegove izbore, ali ne vrše direktno upis u stanje igre. One pozivaju funkcije `choose_game_mode(game_mode_code)`, `choose_a_player(player_code)`, `choose_first_symbol(first_symbol)` i `choose_board_size(board_size)`, koje vrše validaciju unosa i ažuriraju rečnik **game_config**. Ovaj rečnik predstavlja parametre igre, uključujući veličinu table, režim igre, koji igrač igra prvi i koji simbol započinje igru. Ostali delovi koriste vrednosti iz

`game_config` kako bi se inicijalizovao tok igre i osigurala konzistentnost između korisničkog unosa i logike igre.

GUI - prikaz stanja kroz grafičku tablu

Nikola Ristić

Grafički korisnički interfejs (GUI) našeg projekta realizovan je korišćenjem biblioteke **tkinter**, uz upotrebu **PIL (Pillow)** biblioteke za naprednu obradu slika i tekstura. Cilj je bio da se apstraktno stanje grafa verno i interaktivno prikaže korisniku u vidu heksagonalne table.

Za vizuelni prikaz polja ne koristimo obične oblike, već **spritesheet** sistem. Funkcija `load_resources()` učitava `buttons_aqua.png` i vrši isecanje (crop) konkretnih delova slike za stanja **RED**, **GREEN** i **NONE**. Dodatno, koristimo funkciju `darken_image` koja putem `ImageEnhance.Brightness` modula dinamički kreira tamnije varijante polja za potrebe pomoćnih elemenata i vizuelnih efekata (npr. oznaka table). Sve slike se čuvaju u `sprite_cache` rečniku kako bi se izbeglo ponovno učitavanje i procesiranje pri svakom osvežavanju ekrana.

S obzirom na to da je stanje igre u grafu predstavljeno logičkim koordinatama poput `(A, 1)` ili `('LEFT', 1)`, ključni izazov bio je njihovo mapiranje na pikselne koordinate **Canvas** elements. To obavlja funkcija `get_canvas_coords`: ona prepoznaje specijalne kolone **LEFT** i **RIGHT** i dodeljuje im fiksne indekse van standardnog **A-I** opsega. Pozicija se računa u odnosu na centralnu tačku ekrana, koristeći `base_unit` (osnovnu jedinicu mere koja zavisi od trenutne veličine prozora). Heksagonalni raspored se postiže matematičkim pomeranjem: horizontalni razmak je $base_unit \cdot 0.866$, dok se vertikalni korak modifikuje u zavisnosti od kolone kako bi se dobio “cik-cak” efekat saća.

Tabla nije statična, već reaguje na korisničke akcije putem **event binding-a**:

- **Hover efekat:** Funkcija `on_mouse_move` koristi `canvas.find_overlapping` da detektuje preko kog polja se nalazi kursor. Tagovi tipa `coord_A_1` omogućavaju nam da brzo povežemo objekat na platnu sa ključem u rečniku stanja grafa.
- **Promena stanja:** Klikom na polje (`handle_click`), vrši se ciklična promena stanja (**NONE** -> **RED** -> **GREEN** -> **NONE**) unutar same strukture grafa, što se odmah vizuelno reflektuje na ekranu i služi kao privremena simulacija interaktivnosti.
- **Responzivnost:** Funkcija `perform_resize` je zadužena za dinamičko skaliranje svih elemenata. Prilikom promene veličine prozora, sve slike (polja, pozadina...) se ponovo prikazuju korišćenjem `Image.Resampling.BILINEAR` metode radi postizanja responzivnog prikaza.

Pre samog iscrtavanja table, implementiran je meni kroz funkciju `show_setup_menu()`. Ovaj prozor omogućava korisniku da izabere:

- **Veličinu table** - 5, 7 ili 9, što direktno utiče na kalkulaciju u logici grafa.
- **Režim igre** - čovek protiv čoveka ili čovek protiv računara.
- **Prvog igrača** - zeleni ili crveni.

Svi ovi parametri se skladište u `game_config` pre nego što funkcija `draw(state)` pokrene glavni ciklus igre.